

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería
Análisis y Diseño de Algoritmos
Semestre I – 2026



Excelencia que trasciende

DEL VALLE
GRUPO EDUCATIVO

Tarea 1

Integrante:

Nájera Marakovits, Ingrid Nina Alessandra – 231088

Ejercicio 1

Se tiene un arreglo ordenado S y un número x . Se desea decidir si existen dos elementos de S cuya suma sea x .

Como el arreglo está ordenado, se puede recorrer cada elemento $S[i]$ y buscar si existe el complemento $x - S[i]$ usando búsqueda binaria.

Pseudocódigo

```
TwoSum(S, x):  
  para i = 0 hasta n-1:  
    t = x - S[i]  
    si BinarySearch(S, t) return verdadero:  
      return verdadero  
  return falso
```

Complejidad

El ciclo se repite n veces y cada búsqueda binaria tarda $\log n$. Entonces el tiempo total sería:

$$T(n) = n \log n \Rightarrow \Theta(n \log n)$$

Ejercicio 2

El método de Horner hace un ciclo desde $i = n$ hasta 0, por lo que hay $n + 1$ iteraciones. En cada iteración se realizan operaciones constantes, entonces:

$$T(n) = \Theta(n)$$

Ejercicio 3

Para evaluar el polinomio sin usar una operación de potencia, se puede calcular x^k con multiplicaciones repetidas.

Pseudocódigo naïve

```
NaivePoly(a, x):  
  y = 0  
  para k = 0 hasta n:  
    pow = 1  
    para j = 1 hasta k:  
      pow = pow * x  
    y = y + a[k] * pow
```

return y

Complejidad

Para calcular las potencias se realizan:

$$\sum_{k=0}^n k = \frac{n(n+1)}{2} = \Theta(n^2)$$

Por eso el método naïve es $\Theta(n^2)$, mientras que Horner es $\Theta(n)$.

Ejemplo con los input que nos dio en clase

Se elige $x = 2$ y coeficientes cualesquiera, únicamente para ilustrar el conteo de multiplicaciones.

- Horner: multiplicaciones = $n + 1$
- Naïve: multiplicaciones $\approx \frac{n(n+1)}{2} + (n + 1)$

n	Multiplicaciones naïve	Multiplicaciones Horner
4	$10 + 5 = 15$	5
5	$15 + 6 = 21$	6
6	$21 + 7 = 28$	7

Enlace de Git para ver el código funcional: <https://github.com/Ninaswiftie09/ADA>

Acá se muestra el output del programa realizado:

```
PS C:\Users\ninan\OneDrive\Documentos\Escritorio\UVG\VII SEMESTRE\Análisis y Diseño de Algoritmos\ADA> & C:\ProgramData\Python\Python32\python.exe "c:/Users/ninan/OneDrive/Documentos/Escritorio/UVG/VII SEMESTRE/Análisis y Diseño de Algoritmos/ADA.py"
grado n = 4
resultado naive = 73
resultado horner = 73
mults naive (teóricas) : 15
mults horner (teóricas): 5
-----
grado n = 5
resultado naive = 201
resultado horner = 201
mults naive (teóricas) : 21
mults horner (teóricas): 6
-----
grado n = 6
resultado naive = 73
resultado horner = 73
mults naive (teóricas) : 28
mults horner (teóricas): 7
-----
PS C:\Users\ninan\OneDrive\Documentos\Escritorio\UVG\VII SEMESTRE\Análisis y Diseño de Algoritmos\ADA>
```

Figura 1: Salida del programa

Ejercicio 4

Se desea demostrar que $\max(f(n), g(n)) = \Theta(f(n) + g(n))$, asumiendo que f y g son no negativas para n suficientemente grande.

Sea $M(n) = \max(f(n), g(n))$. Se cumple:

$$M(n) \leq f(n) + g(n)$$

Además, como $f(n) \leq M(n)$ y $g(n) \leq M(n)$, entonces:

$$f(n) + g(n) \leq 2M(n)$$

Por lo tanto:

$$M(n) \leq f(n) + g(n) \leq 2M(n)$$

Esto implica que $\max(f(n), g(n)) = \Theta(f(n) + g(n))$.

Ejercicio 5

Se desea demostrar que para constantes reales a y $b > 0$ se cumple $(n + a)^b = \Theta(n^b)$.

Se analiza el cociente:

$$\frac{(n + a)^b}{n^b} = \left(1 + \frac{a}{n}\right)^b$$

Cuando $n \rightarrow \infty$, se tiene $\frac{a}{n} \rightarrow 0$, por lo que el cociente tiende a 1. Entonces:

$$(n + a)^b = \Theta(n^b)$$

Ejercicio 6

Acá vamos a analizar caso por caso.

Primer caso

$$2^{n+1} = 2 \cdot 2^n$$

Como solo cambia por un factor constante, se cumple:

$$2^{n+1} = O(2^n)$$

Segundo caso

$$2^{2n} = (2^n)^2 = 4^n$$

Se compara con 2^n :

$$\frac{2^{2n}}{2^n} = 2^n \rightarrow \infty$$

Por lo tanto:

$$2^{2n} \neq O(2^n)$$

Ejercicio 7

Parte a

Se desea demostrar que $f = \Theta(g)$ si y solo si $f = O(g)$ y $f = \Omega(g)$.

Por definición, $f = \Theta(g)$ significa que existen $c_1, c_2 > 0$ y n_0 tales que:

$$c_1 g(n) \leq f(n) \leq c_2 g(n) \quad \text{para } n \geq n_0$$

La desigualdad superior corresponde a $f = O(g)$ y la inferior corresponde a $f = \Omega(g)$. Si se cumplen ambas, entonces se cumple $f = \Theta(g)$.

Parte b

Se desea demostrar que $o(g) \cap \omega(g) = \emptyset$.

Si $f \in o(g)$, entonces $\frac{f(n)}{g(n)} \rightarrow 0$. Si $f \in \omega(g)$, entonces $\frac{f(n)}{g(n)} \rightarrow \infty$. No es posible que un mismo cociente tienda a 0 e infinito al mismo tiempo, por lo tanto la intersección es vacía.

Parte c

Se desea demostrar que si $f = O(g)$ entonces $\log_2 f = O(\log_2 g)$, asumiendo que $f \geq 1$ y $\log_2 g \geq 1$ para n suficientemente grande.

Si $f = O(g)$, existe una constante $c > 0$ tal que $f(n) \leq c g(n)$ para n grande. Aplicando logaritmo base 2:

$$\log_2 f(n) \leq \log_2(c g(n)) = \log_2 c + \log_2 g(n)$$

Como $\log_2 g(n) \geq 1$ para n grande, la suma anterior queda acotada por una constante multiplicada por $\log_2 g(n)$. Entonces:

$$\log_2 f(n) = O(\log_2 g(n))$$